

Trilha de Aprendizado AdonisJS — Padrão Interno para Empresa Júnior

TL;DR

- A trilha deve ser construída sobre o **AdonisJS v7** (versão estável atual, lançada em 25/02/2026; release estável mais recente v7.3.4, publicada em 05/06), usando o **starter kit oficial "API"** como base para devs que vêm de Express e vão trabalhar com APIs REST; os pré-requisitos oficiais são **Node.js ≥ 24.x e npm ≥ 11.x** (o Node.js 24 tornou-se LTS em outubro de 2025).
- O caminho essencial de aprendizado, na ordem lógica, é: Visão geral → Instalação/CLI Ace → Estrutura de pastas/IOC → Config e .env → Routing → Controllers → Middleware → Validação (VineJS) → Lucid ORM (migrations, models, relationships, seeders) → Autenticação (guards) → Testes (Japa) → **Projeto final: API REST CRUD com autenticação**.
- Não existe canal oficial do AdonisJS no YouTube; a documentação oficial (docs.adonisjs.com) e o tutorial oficial "DevShow" são as fontes primárias, e a documentação recomenda os screencasts "**Let's Learn AdonisJS**" do Adocasts (**parceiro, não oficial**) como material em vídeo.

Key Findings

1. Versão e posicionamento. O AdonisJS v7 é a versão estável atual e, segundo o blog oficial ("AdonisJS v7 is here"), "is an incremental upgrade to v6 with minimal breaking changes" — o lançamento atualizou 45+ pacotes oficiais e introduziu 3 novos (`@adonisjs/otel`, `@adonisjs/content` e `edge-markdown`). É um framework "batteries-included", TypeScript-first, ESM, com arquitetura MVC e um ecossistema de pacotes de primeira parte (Lucid ORM, Auth, VineJS, Japa, Edge). O diferencial central frente ao Express é: em vez de montar dezenas de pacotes npm, o dev recebe roteamento, middleware, validação, ORM, autenticação e testes já integrados e tipados.

2. Fontes oficiais em vídeo são limitadas. Não há canal oficial no YouTube; a própria documentação delega o aprendizado em vídeo ao Adocasts. Portanto, a trilha deve ser ancorada na documentação oficial escrita e no tutorial oficial "DevShow", com os screencasts do Adocasts como complemento opcional claramente marcado como parceiro.

3. O caminho é fortemente convencional. Os comandos `node ace make:*` geram controllers, models, migrations, validators, middleware e testes já no lugar certo, o que torna a curva de um dev júnior que já conhece Express bastante suave. Segundo o blog oficial, a migração de v6 para v7 é rápida ("We expect most apps to migrate to v7 within 30 minutes to an hour"), o que sinaliza estabilidade das APIs.

Details

Módulo 0 — Visão geral do AdonisJS (~1-2 h)

O que é: AdonisJS é um framework backend para Node.js, escrito em TypeScript, que roda em modo ESM e oferece type-safety de ponta a ponta. Ele fornece os blocos essenciais para construir e manter backends completos, eliminando a necessidade de serviços de terceiros para recursos comuns como autenticação, upload de arquivos, cache e rate limiting.

Versão atual: AdonisJS v7 (estável, lançado em 25/02/2026, tag v7.0.0 no GitHub — "Type-safe URL builder, barrel files, tooling for e2e type-safety and much more"). O v6 continua documentado e é migrável para v7 em 30 minutos a 1 hora, segundo a equipe. Evite conteúdos de Adonis 4/5 exceto para contextualizar a evolução (v5 usava `@ioc` e compilador TS customizado; v6 trouxe ESM, IoC simplificado e VineJS; v7 trouxe type-safety E2E, novos starter kits e OpenTelemetry).

Diferenciais frente a Express/NestJS/Laravel-Rails: Em relação ao Express/Fastify, o AdonisJS fornece estrutura e convenções que o Express não tem, mantendo desempenho comparável — em vez de montar pacotes você mesmo, recebe um kit integrado. Em relação ao NestJS, foca em padrões práticos em vez de abstrações "enterprise". Em relação a Laravel/Rails, traz a mesma experiência coesa (migrations, seeders, factories, relacionamentos de modelo, convenções consistentes) para o mundo Node.js.

Links oficiais:

- Introdução: <https://docs.adonisjs.com/introduction>
- Blog "AdonisJS v7 is here": <https://adonisjs.com/blog/v7>
- Site oficial: <https://adonisjs.com/>
- Org no GitHub: <https://github.com/adonisjs>

Módulo 1 — Pré-requisitos técnicos (~1 h)

Versões: conforme a doc oficial de instalação, **Node.js ≥ 24.x e npm ≥ 11.x** (verificar com `node -v` e `npm -v`). O blog do v7 reforça: "v7 requires Node.js 24 or above. As of October 2025, Node.js 24 became the LTS version." Recomenda-se um gerenciador de versões como Volta ou nvm.

Conhecimentos prévios recomendados pela doc oficial: familiaridade com o ecossistema Node.js e programação assíncrona; noções de TypeScript (a doc recomenda aprender o básico de TS antes de usar AdonisJS). O público-alvo desta trilha (devs que já sabem Node/Express, REST e JS/TS) atende a esses pré-requisitos. `AdonisJS`

Links oficiais:

- Instalação/pré-requisitos: <https://docs.adonisjs.com/installation>
- Setup de ambiente de desenvolvimento: <https://docs.adonisjs.com/dev-environment>

Módulo 2 — Instalação e criação do projeto (~1-2 h)

Comando oficial de scaffolding:

```
npm create adonisjs@latest [nome-do-projeto]
```

Isso baixa o pacote inicializador `create-adonisjs` e inicia um setup interativo. Flags úteis:

- `--kit`: escolhe o starter kit (`hypermedia`, `react`, `vue`, `api`)
- `--db`: dialeto do banco (`sqlite`, `postgres`, `mysql`, `mssql`)
- `--auth-guard`: guard de autenticação (`session`, `access_tokens`, `basic_auth`)
- `--git-init`: inicializa repositório git

Exemplo recomendado para a empresa (API + Postgres + tokens):

```
npm create adonisjs@latest minha-api -- --kit=api --db=postgres --auth-guard=access_tokens
```

Depois: `cd minha-api` e iniciar o servidor de desenvolvimento com `node ace serve --watch` (o backend roda em <http://localhost:3333>).

Starter kits oficiais (quatro): Hypermedia (Edge + Alpine.js), React (Inertia + React), Vue (Inertia + Vue) e API (monorepo com Turborepo, backend AdonisJS + frontend à escolha, com type-safety E2E). Todo projeto novo já inclui estrutura de pastas opinativa, Lucid ORM configurado com SQLite por padrão, fluxos de login/signup prontos, e ESLint + Prettier.

Links oficiais:

- Instalação: <https://docs.adonisjs.com/installation>
- Pick your path (comparação de kits): <https://docs.adonisjs.com/stacks-and-starter-kits>
- Starter kits no GitHub: <https://github.com/adonisjs/starter-kits>

Módulo 3 — Estrutura de pastas, Ace CLI e IoC container (~2-3 h)

Estrutura padrão:

```

├─ app/           (controllers, middleware, models, validators, exceptions, mails)
├─ bin/           (console.ts, server.ts, test.ts – entrypoints)
├─ config/        (arquivos de configuração runtime)
├─ database/      (migrations, seeders, factories, schema.ts)
├─ resources/     (templates Edge / assets)
├─ start/         (routes.ts, kernel.ts, env.ts – carregados no boot)
├─ tests/         (unit, functional)
├─ adonisrc.ts    (manifesto do projeto)
├─ tsconfig.json
└─ package.json  (import aliases com subpath imports, ex.: #models/*, #controllers/*)

```

O `adonisrc.ts` é o manifesto que diz ao AdonisJS como descobrir e carregar partes da aplicação (providers, comandos, arquivos a pré-carregar).

Ace CLI: framework de linha de comando embutido, executado via `node ace <comando>`. Usado para gerar recursos (`make:controller`, `make:model`, `make:migration`, `make:validator`, `make:middleware`, `make:test`), rodar migrations, abrir REPL e listar rotas (`node ace list:routes`).

IoC container / Injeção de dependência: O AdonisJS traz um IoC container zero-config que usa reflection para resolver e injetar dependências de classes. Ao type-hint uma classe como dependência, o container cria a instância automaticamente. Usa-se o decorator `@inject()` para resolução automática. O container já está integrado em controllers, middleware, event listeners e comandos Ace. Um dev júnior precisa entender o conceito de "container services" (imports prontos como `import router from '@adonisjs/core/services/router'`) e o `@inject`, mas não precisa dominar providers customizados de início.

Links oficiais:

- Estrutura de pastas: <https://docs.adonisjs.com/folder-structure>
- Injeção de dependência / IoC: <https://docs.adonisjs.com/guides/concepts/dependency-injection>
- Container services: <https://docs.adonisjs.com/guides/concepts/container-services>

Módulo 4 — Configuração e variáveis de ambiente (~1-2 h)

Configuração fica em `config/`; cada arquivo exporta um objeto (ex.: `config/database.ts`, `config/auth.ts`). Variáveis de ambiente ficam em `.env`, com validação e type-safety definidas em `start/env.ts` via `Env.create` — se uma variável obrigatória faltar, a aplicação recusa iniciar. O `APP_KEY` (usado para criptografia de cookies e assinatura de sessões) é gerado com `node ace generate:key`. Suporta múltiplos arquivos `.env` por ambiente (`.env`, `.env.test`, etc.).

Links oficiais:

- Configuração & Environment: <https://docs.adonisjs.com/configuration>

- Variáveis de ambiente (guia): <https://docs.adonisjs.com/guides/getting-started/environment-variables>

Módulo 5 — Routing (~2 h)

Rotas ficam em `start/routes.ts`. Uma rota combina um padrão de URI e um handler.

Métodos: `router.get/post/put/patch/delete`, `router.any`, `router.route`. Route params (`:id`), params opcionais (`:id?`), wildcard (`*`), e matchers/validação de params via `.where()`.

Rotas podem referenciar controllers de forma type-safe via `#generated/controllers`. Grupos (`router.group`) com prefixos e middleware. `router.resource()` gera as sete rotas RESTful de CRUD automaticamente.

Link oficial: <https://docs.adonisjs.com/guides/basics/routing>

Módulo 6 — Controllers (~2 h)

Controllers organizam handlers em classes dedicadas em `app/controllers`. Criados com `node ace make:controller`. São instanciados por requisição usando o IoC container (permite injeção de dependências). Ações RESTful seguem convenções; `router.resource('posts', controllers.Posts)` mapeia CRUD. Recomenda-se ao júnior aprender single-action e resource controllers.

Link oficial: <https://docs.adonisjs.com/guides/basics/controllers>

Módulo 7 — Middleware (~2 h)

Middleware executa durante a requisição HTTP, antes de chegar ao handler. Baseado no padrão Chain of Responsibility, com fase "downstream" (antes do `next()`) e "upstream" (depois). Três stacks: **server** (roda em toda requisição, mesmo sem rota), **router/global** (toda requisição com rota — ex.: bodyparser, session, auth) e **named** (aplicado seletivamente a rotas). Criado com `node ace make:middleware`; registrado em `start/kernel.ts`. Named middleware pode receber configuração (ex.: `middleware.auth({ guards: [...] })`).

Link oficial: <https://docs.adonisjs.com/guides/basics/middleware>

Módulo 8 — Validação com VineJS (~2-3 h)

VineJS é a biblioteca oficial de validação (framework-agnostic), criada pela equipe AdonisJS. Segundo a documentação oficial do VineJS (vinejs.dev), "VineJS is one of the fastest validation libraries in the Node.js ecosystem", com performance de "5x-10x better than Zod" na validação de corpo de requisições HTTP; traz type-safety estática junto com validação em runtime. Vem pré-configurada nos starter kits web e api. Cria-se um validator por ação (ex.:

`createPostValidator`, `updatePostValidator`) com `node ace make:validator`, usando `vine.compile(vine.object({...}))`. No controller, usa-se `request.validateUsing(validator)` — não é preciso try/catch, pois o AdonisJS converte erros de validação em resposta HTTP automaticamente (com content negotiation). O Lucid estende o VineJS com regras `unique` e `exists` para validação contra o banco.

Links oficiais:

- Validação (guia): <https://docs.adonisjs.com/guides/basics/validation>
- Regras de banco (unique/exists): <https://lucid.adonisjs.com/docs/validation>
- Site do VineJS: <https://vinejs.dev>

Módulo 9 — Lucid ORM (~4-6 h, o módulo mais denso)

Lucid é o ORM SQL oficial, um Active Record construído sobre o Knex, com suporte a PostgreSQL, MySQL, SQLite, MSSQL e outros. Subtópicos essenciais:

- **Migrations:** definem a estrutura do banco; criadas com `node ace make:migration` (ou junto do model com `make:model -m`). Classes estendem `BaseSchema` com métodos `up()` e `down()`. Executadas com `node ace migration:run`; rollback com `migration:rollback`; `migration:fresh --seed` recria o banco. O AdonisJS rastreia migrations executadas na tabela `adonis_schema`. (AdonisJS)
- **Models:** no v7 seguem abordagem "migrations-first" — após rodar migrations, o Lucid escaneia o banco e gera classes de schema tipadas em `database/schema.ts`; seu model estende essa classe gerada e adiciona apenas relacionamentos, hooks e lógica de negócio. Criados com `node ace make:model`.
- **CRUD e query builder:** `Model.create`, `Model.find`, `Model.query()`, `db.from('table')`.
- **Relacionamentos:** `@hasOne`, `@hasMany`, `@belongsTo`, `@manyToMany` (com pivot table), `@hasManyThrough`; carregamento com `preload`.
- **Seeders e factories:** seeders populam o banco (`node ace make:seeder`, `node ace db:seed`); factories geram dados de teste.

Links oficiais:

- Site do Lucid: <https://lucid.adonisjs.com/>
- Introdução/Models: <https://lucid.adonisjs.com/docs/models>
- Relacionamentos: <https://lucid.adonisjs.com/docs/relationships>
- Guia SQL ORM na doc principal: <https://docs.adonisjs.com/guides/database/lucid>

Módulo 10 — Autenticação (~3-4 h)

O pacote `@adonisjs/auth` é construído em torno de **guards** (implementações de um tipo de login) e **providers** (buscam usuários/tokens no banco). Guards embutidos:

- **session:** para apps server-rendered ou API no mesmo top-level domain (usa cookies/sessão).
- **access_tokens:** tokens opacos (Opaque access tokens), prefixo `oat_`, hash armazenado no banco — para apps mobile ou frontend em domínio diferente; permite revogação instantânea.
- **basic_auth:** HTTP Basic (uso temporário em dev).

Configuração em `config/auth.ts`. Setup com `node ace add @adonisjs/auth --guard=<session|access_tokens|basic_auth>`, que cria model User, migration de users e middleware. Verificação de credenciais via `User.verifyCredentials` (mixin `AuthFinder`, protege contra timing attacks); hashing de senha automático. Proteção de rotas com `middleware.auth()`. O escopo do pacote é autenticar requisições — registro de usuário, recuperação de senha e RBAC/permisões ficam fora (para autorização há o Bouncer).

Links oficiais:

- Autenticação (introdução): <https://docs.adonisjs.com/guides/auth>
- Session guard: <https://docs.adonisjs.com/guides/auth/session-guard>
- Access tokens guard: <https://docs.adonisjs.com/guides/auth/access-tokens-guard>

Módulo 11 — Testes com Japa (~3 h)

Japa é o test runner oficial. Segundo a documentação oficial (docs.adonisjs.com/guides/testing/introduction), "The testing layer is powered by Japa, a testing framework we've built and maintained for over seven years. Unlike general-purpose test runners like Jest or Vitest, Japa is purpose-built for backend applications." Ele roda nativamente em Node.js sem transpilers e integra com o AdonisJS via `@japa/plugin-adonisjs`. Testes organizados em suites (`unit` e `functional`). Criados com `node ace make:test`; executados com `node ace test` (flags: `--watch`, `--tags`, `--files`). Testes de API usam o API client do Japa (`client.get/post/...` ou `client.visit(routeName)`), que sobe o servidor HTTP e faz requisições reais. O pacote de auth adiciona `loginAs` para autenticar nos testes; sessões usam driver `memory` em `.env.test`.

Links oficiais:

- Testes (introdução): <https://docs.adonisjs.com/guides/testing/introduction>
- Testes de API: <https://docs.adonisjs.com/guides/testing/api-tests>
- Site do Japa: <https://japa.dev>

Recursos complementares oficiais

- **Tutorial oficial "DevShow"** (site community showcase) na documentação, com versões Hypermedia e React: Overview, CLI & REPL, Database and models, Routes/controllers/views, Forms and validation, Styling and cleanup, Authorization. <https://docs.adonisjs.com/tutorial/hypermedia/overview>
- **Starter kits oficiais:** <https://github.com/adonisjs/starter-kits>
- **Referência da API/Ace:** <https://docs.adonisjs.com/reference/application>
- **Screencasts recomendados pela doc (parceiro, NÃO oficial):** "Let's Learn AdonisJS 6/7" do Adocasts (Tom Gobich) — <https://adocasts.com/series/lets-learn-adonisjs-6> e <https://adocasts.com/series/lets-learn-adonisjs-7>; playlist no YouTube da série 6: <https://www.youtube.com/playlist?list=PL9dIWIKCV573Pfg9aaRO8zogHYPaWTLyG>

- **Canais oficiais de comunidade:** Discord, X/Twitter @adonisframework, GitHub Discussions.

(a) Resumo dos módulos com links (para o documento Word/Markdown)

Ver a seção Details acima — cada módulo já traz descrição + links oficiais. Estrutura recomendada de tempo total: ~28-38 horas de estudo, distribuídas em ~2-3 semanas em regime de meio período.

(b) Fluxo/etapas da trilha (para gerar o diagrama/roadmap)

Sequência linear com dependências (cada módulo depende do anterior, salvo indicação):

1. **Visão Geral** → base conceitual.
2. **Pré-requisitos** (Node 24+, TS básico) → depende de (1).
3. **Instalação & CLI Ace** → depende de (2).
4. **Estrutura de Pastas & IoC** → depende de (3).
5. **Config & .env** → depende de (4).
6. **Routing** → depende de (5).
7. **Controllers** → depende de (6).
8. **Middleware** → depende de (7).
9. **Validação (VineJS)** → depende de (7); relaciona-se com (10) via regras unique/exists.
10. **Lucid ORM** (migrations → models → relationships → seeders) → depende de (5) e (7).
11. **Autenticação (guards)** → depende de (8), (9) e (10).
12. **Testes (Japa)** → depende de (7)-(11).
13. **PROJETO FINAL — API REST** → integra (6)-(12).

Blocos do roadmap sugeridos: **Fundamentos** (1-5) → **Camada HTTP** (6-8) → **Dados & Validação** (9-10) → **Segurança & Qualidade** (11-12) → **Projeto Final** (13).

Recommendations

Estágio 1 — Preparar o ambiente-padrão (semana 0). Padronize Node.js ≥ 24.x via Volta/nvm, VS Code com as extensões oficiais AdonisJS e Edge, e um projeto-modelo criado com `npm create adonisjs@latest treino -- --kit=api --db=postgres --auth-guard=access_tokens`. Benchmark para avançar: o novo dev consegue subir o servidor (`node ace serve --watch`) e ver a resposta JSON em <http://localhost:3333>.

Estágio 2 — Fundamentos guiados (semana 1). Módulos 0-5, lendo a doc oficial e reproduzindo comandos Ace. Benchmark: o dev explica a estrutura de pastas, cria um controller e uma rota, e valida uma variável de ambiente em `start/env.ts`.

Estágio 3 — Camada HTTP + dados (semana 2). Módulos 6-10, culminando em um mini-CRUD (ex.: uma entidade "task") com migration, model, validator e resource controller. Benchmark: CRUD completo funcionando com validação VineJS e persistência via Lucid.

Estágio 4 — Segurança, testes e projeto final (semana 3). Módulos 11-12 e o projeto guiado. Benchmark: API final com autenticação por token, rotas protegidas e ao menos 3 testes funcionais passando (`node ace test`).

Projeto prático final guiado — "API de Gerenciamento de Tarefas" (Task Manager API).

Escopo alinhado à trilha:

- **Entidades:** `User` (do starter kit) e `Task` (título, descrição, status, `user_id`).
- **Relacionamento:** `User hasMany Task` / `Task belongsTo User` (Módulo 9).
- **Migrations + Model + Seeder** para `tasks` (Módulo 9).
- **CRUD completo** via `router.resource('tasks', controllers.Tasks)` (Módulos 5-6).
- **Validação** com `createTaskValidator` / `updateTaskValidator` no VineJS (Módulo 8).
- **Autenticação** com guard `access_tokens`: endpoints de registro/login emitindo token, e `middleware.auth()` protegendo as rotas de tasks, com escopo por usuário (Módulo 10).
- **Testes funcionais** com Japa + API client, usando `loginAs` para testar rotas protegidas (Módulo 11). Este projeto espelha o tutorial oficial "DevShow" (posts/comentários com autenticação e autorização), adaptado para uma API pura de tarefas.

Thresholds que mudam a recomendação: se a empresa for construir apps full-stack renderizados no servidor, troque o kit `api` pelo `hypermedia` e priorize o guard `session` + Edge; se for usar React/Vue com Inertia, use os kits `react` / `vue`.

Caveats

- **Versão em transição:** o v7 é recente (fev/2026). Parte do conteúdo em vídeo de terceiros ainda cobre v6 (ex.: "Let's Learn AdonisJS 6"). As diferenças v6→v7 são pequenas (a equipe estima migração em 30 min a 1 h), mas ao seguir vídeos antigos o dev pode notar divergências (especialmente na abordagem migrations-first de models do Lucid e nos caminhos de doc de auth).
- **Sem vídeos oficiais:** confirmado que não há canal oficial do AdonisJS no YouTube; os screencasts recomendados são do Adocasts (parceiro/comunidade, não first-party). Trate-os como complemento, não como fonte canônica.
- **Estimativas de tempo** são aproximadas e dependem da experiência prévia do dev com TypeScript e ORMs; ajuste conforme o desempenho nos benchmarks de cada estágio.
- **Escopo do pacote de auth:** ele autentica requisições, mas não cobre registro/recuperação de senha/RBAC — esses precisam ser implementados manualmente (ou via Bouncer para autorização), o que deve ser deixado claro no projeto final.
- Alguns caminhos de URL da documentação de autenticação aparecem em duas formas (`/guides/auth/...` e `/guides/authentication/...`) conforme a versão; use sempre o índice em <https://docs.adonisjs.com> como referência canônica.
- **Detalhe de versão:** a tag v7.0.0 foi publicada em 25/02/2026 no GitHub; a release estável mais recente identificada é a **v7.3.4 (05/06)**. Verifique <https://github.com/adonisjs/core/releases> para a versão vigente no momento de montar o material.

